KV Forge: Mastering LLM Inference, from KV Cache to Production — Basic

Portfolio projects · lessons 01–09

Each takes 1–3 days and ends with something to show an employer.

Project 1: Brightside capacity report: Llama 3.1 8B on one L4

Goal: Size one 8B model on one 24 GB GPU from its config and a measured latency target, so the team can say how many support requests a single card carries before it misses its TTFT SLO.

Uses lessons: 03, 04, 05, 06, 09

1. Read the Llama 3.1 8B architecture numbers (32 layers, 8 KV heads, head_dim 128) from the lesson 03 table and compute bytes per token as 2 x layers x KV heads x head_dim x 2 bytes; check you get 128 KiB per token.

2. Compute the cost of one Brightside request at full length: 1,500 policy tokens plus 300 question tokens is 1,800 tokens, so 1,800 x 128 KiB = 225 MiB of KV per request.

3. Build the memory budget on an L4 (24 GB): FP16 weights of about 15 GiB, 0.92 gpu_memory_utilization, and a stated allowance for activations; the rest is the KV budget.

4. Divide the KV budget by 225 MiB to get the concurrency limit; write the arithmetic in a table with each term on its own line, and expect roughly 15 to 20 requests.

5. Run the lesson 09 notebook on the CPU image with Qwen2.5-0.5B-Instruct, or load the recorded result in data/09/report.json, and measure p50 and p95 TTFT at concurrency 1, 2 and 4 with 12 requests each.

6. Set your own TTFT p95 target in seconds, state which measured concurrency meets it, and write the capacity claim as one sentence: 'one L4 holds N requests at p95 TTFT under X s, measured on Y'.

7. Write the report as report.md with the formulas, the table, the measured rows and the list of what was estimated rather than measured.

Deliverable: A git repo with a README.md that holds the capacity report (estimate versus measured, with the recorded run named and its date), the notebook or script that produced each number, and a results table with p50 and p95 TTFT per concurrency level.

Stretch: Repeat the sizing for Llama 3.2 3B (112 KiB per token) and show which model fits more requests at the same SLO, and why the answer is not just the smaller model.

Project 2: Support desk latency and goodput tester

Goal: Build a small streaming client that replays the Brightside support trace against an OpenAI-compatible server and reports TTFT, TPOT and goodput against an SLO, the numbers an on-call engineer checks first.

Uses lessons: 06, 07, 08

1. Start the OpenAI-compatible server from lesson 08 (CPU image or the recorded run) and keep its API key in .env, printing only bool(value) in logs.

2. Load data/07/support_trace.jsonl: 30 requests with prompt tokens of about 1,500 to 1,600 and output tokens between 24 and 113; check the min, max and mean of both columns before you run anything.

3. Stream each request with the openai client, stamp the time of the first token and the last token, and compute TTFT and TPOT as (last - first) / (output tokens - 1) for each request.

4. Set an SLO of your choice (for example TTFT at most 1.5 s and TPOT at most 60 ms) and compute goodput as the share of requests that meet both at once, not each one separately.

5. Run the replay at concurrency 1 and 2 with a thread pool, and record p50 and p95 TTFT, median TPOT, total output tokens per second and goodput for each run.

6. Write a one-page note: which number breached first as concurrency rose, and whether the breach came from TTFT (prefill queueing) or TPOT (decode speed).

Deliverable: A git repo with the replay script, a README with a results table (p50 and p95 TTFT, median TPOT, tokens per second, goodput at each concurrency), the SLO written at the top, and the one-page note on the first breach.

Stretch: Add a retry-free timeout of 30 s and count how many requests time out at concurrency 4, then explain whether timeouts or slow first tokens are the bigger risk.

Project 3: Tenant-rights advice desk: how long can a conversation get

Goal: Size the KV memory of long multi-turn advice conversations for a housing nonprofit's assistant and choose a history cap backed by measured prefill time, so long cases do not silently slow the desk for everyone.

Uses lessons: 02, 03, 04, 05, 06, 08

1. Assume a case where each turn adds about 250 tokens of user and assistant text and the shared policy prompt is 1,500 tokens; a 20-turn case is 1,500 + 20 x 250 = 6,500 tokens.

2. From the lesson 03 config table compute the KV for 6,500 tokens for three models: Qwen2.5-0.5B (12 KiB per token, about 76 MiB), Llama 3.2 3B (112 KiB, about 711 MiB) and Llama 3.1 8B (128 KiB, about 812.5 MiB); show each multiplication.

3. Use lesson 04 to explain in two sentences how many KV heads the 8B model keeps (8, grouped-query) against the 32 of a full multi-head design, the 4x saving that buys, and what grouping gives up.

4. Turn the KV numbers into a concurrency limit on a 24 GB GPU with the lesson 05 budget, for a 3B model and an 8B model, at history lengths of 5, 10, 20 and 40 turns.

5. Start the server from lesson 08 and time the time to first token for prompts of 2,000, 4,000, 6,500 and 8,000 tokens, five runs each; record the median for each length.

6. Fit a straight line to prefill time against prompt length from the medians, and write the history cap as the turn count where the predicted TTFT is still under the desk's target you set, with the fit reported.

Deliverable: A git repo with the KV table, the concurrency table at each history length, the timing medians, the fitted line with its slope, and a README that recommends one history cap in turns and states the evidence behind it.

Stretch: Add a rule that drops the oldest turns after the cap and keeps the policy prompt, then measure the TTFT change on the same case.

Project 4: kvbudget: a KV memory calculator with tests for the platform team

Goal: Write a small command-line tool that reads a model config and a GPU and prints bytes per token, KV per request and max concurrency, with tests that pin the numbers the course teaches, so the team stops re-doing this arithmetic in spreadsheets.

Uses lessons: 03, 04, 05

1. Create a Python 3.12 project with its own .venv and a pinned requirements.txt; install only pytest and the standard library, and keep API keys out of the repo.

2. Write kv_bytes_per_token(layers, kv_heads, head_dim, dtype_bytes) and test it: Llama 3.1 8B must give 131,072 bytes (128 KiB) and Llama 3.1 70B must give 327,680 bytes (320 KiB).

3. Add an MHA, MQA and GQA comparison: compute the same model with kv_heads equal to 32, 1 and 8 and print the ratio, so the cost of each attention design is visible.

4. Implement max_concurrency(gpu_gib, util, weights_gib, activations_gib, tokens_per_request, bytes_per_token) and test it against the worked example: an L4 with util 0.92 and 8B FP16 weights.

5. Add an FP8 flag that halves dtype_bytes, test that it halves the answer, and print a table for 8B and 70B on an L4 and an 80 GB card with 1,800 tokens per request.

6. Run pytest and record the count of passing tests and the runtime in the README; fail the build if any number in the table changes without a test change.

Deliverable: A git repo with the tool, at least eight pytest tests that pin the course numbers, a requirements file, and a README with the sample output table and the pytest result line.

Stretch: Read a real config.json from a model you choose, check the field names against the lesson 03 formula, and document any model whose head_dim is missing from the config and must be derived.